

Laboratórios de Informática I

2025/2026

Licenciatura em Engenharia Informática

Sistemas de Controlo de Versões

O desenvolvimento de *software* é cada vez mais complexo, e obriga a que uma equipa de programadores possa desenvolver uma mesma aplicação ao mesmo tempo, sem se preocuparem com os detalhes do que outros membros dessa mesma equipa estejam a fazer. Alterações concorrentes (realizadas por diferentes pessoas ao mesmo tempo) podem provocar conflitos quando várias pessoas editam o mesmo bocado de código.

Além disso, não nos devemos esquecer que algumas alterações a um programa, no sentido de corrigir ou introduzir alguma funcionalidade, podem elas mesmas conter erros, e pode por isso ser necessário repor uma versão prévia da aplicação, anterior a essa alteração.

Para colmatar estes problemas são usados sistemas de controlo de versões.

1 Panorama nos Sistemas de Controlo de Versões

Existe um grande conjunto de sistemas que permitem o desenvolvimento cooperativo de *software*. Todos eles apresentam diferentes funcionalidades mas os seus principais objetivos são exatamente os mesmos.

Habitualmente divide-se este conjunto em dois, um conjunto de sistemas denominados de *centralizados*, e um outro de sistemas *distribuídos*:

- Sistemas de controlo de versões centralizados:
 - Concurrent Versions System (CVS): <http://www.nongnu.org/cvs/>;
 - Subversion (SVN): <https://subversion.apache.org/>;
- Sistemas de controlo de versões distribuídos:
 - Git: <http://git-scm.com/>;
 - Mercurial (hg): <http://mercurial.selenic.com/>;
 - Bazaar (bzt): <http://bazaar.canonical.com/en/>;

Estes são apenas alguns exemplos dos mais usados. A grande diferença entre os centralizados e os distribuídos é que, nos centralizados existe um repositório, denominado de servidor, que armazena, a todo o momento, a versão mais recente do código fonte. Por sua vez, nos distribuídos, cada utilizador tem a sua própria cópia do repositório, que pode divergir, havendo posteriormente métodos para juntar repositórios distintos.

2 Instalação do Git

Na disciplina de Laboratórios de Informática I será utilizado o sistema **Git**. Para o instalar siga as instruções em <https://git-scm.com/downloads>.

3 Configurar o git

Para configurar o **Git** ao nível do sistema deve indicar o nome e o email que ficarão associados às actividades realizadas no repositório.

```
$ git config --global user.name "a999999"
```

Configura o nome que irá ficar associado aos git commits, **a999999** neste exemplo.

```
$ git config --global user.email "a999999@alunos.uminho.pt"
```

Configura o email que irá ficar associado aos git commits, **a999999@alunos.uminho.pt** neste exemplo.

Poderá consultar a configuração actual com:

```
git config --list
```

4 Uso do Git

Os ficheiros de uma directoria de trabalho controlada pelo **Git** podem estar no estado *tracked* (fazem parte do repositório ou estão na área de preparação (*staged area*) para que possam ser incluídos no repositório) ou *untracked* (detectadas pelo **Git**, mas desconhecidos do repositório). Os ficheiros no estado *tracked* podem estar *unmodified* (actualizados no repositório), *modified* (modificados desde a última actualização do repositório), or *staged* (marcados como preparados para inclusão no repositório). Para actualizar o repositório, os ficheiros no estado *tracked - staged* terão de ser convertidos em *tracked - unmodified*. Vamos de seguida analisar como o **Git** gere este ciclo de transformações.

4.1 Clone

A primeira etapa no uso do *git* corresponde a criar um repositório local. **Este processo só deverá ser efetuado uma vez.**

Pode criar um repositório vazio com o comando **git init**. Ao invés disso, vamos fazer uma cópia do repositório do trabalho prático, com o comando **git clone**. Para tal, pode executar o comando, substituindo **XXX** pelo número correto do seu grupo.

```
$ git clone git@gitlab.com:uminho-di/li1/2526/projetos/202511gXXX.git
```

O resultado da execução deste comando é uma nova pasta, criada na directoria actual, com o nome do repositório (**202511gXXX** no exemplo acima). Esta pasta será a raiz do repositório. O repositório de cada grupo foi previamente preenchido com pastas e ficheiros.

4.2 Adição de novas directorias e ficheiros

O passo seguinte corresponde a adicionar novos ficheiros ou pastas que queiramos armazenar no repositório. Como exemplo, vamos adicionar um ficheiro com os membros do grupo. Crie dentro da directoria do repositório ficheiro chamado **CREDITOS.md**, e escreva no ficheiro o seu número de aluno e nome completo. Este ficheiro ainda não faz parte do repositório (está no estado *untracked*).

Um comando extremamente simples, mas bastante útil, designado por **status**, permite ver o estado atual do repositório local (sem realizar qualquer ligação ao servidor)

```
$ git status
On branch master
```

```
No commits yet
```

```
Untracked files:
```

```
(use "git add <file>..." to include in what will be committed)
    CREDITOS.md
```

```
nothing added to commit but untracked files present (use "git add" to track)
```

Isto indica que o Git detecta o ficheiro **CREDITOS.md**, mas não sabe nada sobre ele.

Para adicionar o ficheiro **CREDITOS.md** ao repositório terá de começar por executar o comando:

```
$ git add CREDITOS.md
```

O ficheiro foi adicionado à área de preparação para que possa ser incluído no repositório, mas ainda não faz parte do repositório controlado pelo Git. Como veremos, tal só acontecerá quando for executado o comando *commit*.

Se executar:

```
$ git status
On branch master
No commits yet
Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
    new file:   CREDITOS.md
```

a mensagem indica que há um novo ficheiro pronto a ser inserido no repositório. Indica também forma de o remover da *staged area*.

Sempre que realizar alterações a um ficheiro e as quiser registar, terá de dar essa informação ao Git. O mesmo comando também é usado para adicionar novos ficheiros ao repositório. Assim, depois de terminar as alterações a um ficheiro (novo ou não), deve executar o comando **git add**.

Uma boa prática de desenvolvimento é adquirir o hábito de gerir todo o código que programar para o projeto através do sistema de versões.

4.3 Commit

Para registar as alterações e os novos ficheiros ou pastas no repositório local é necessário realizar um processo designado por *commit* ¹. Isto poderá ser feito através do comando `git commit`.

```
$ git commit -m "Adicionado o ficheiro CREDITOS.md"
[master (root-commit) 72e94ad] Adicionado o ficheiro CREDITOS.md
1 files changed, 4 insertions(+)
create mode 100644 CREDITOS.md
```

No comando *commit* executado foi adicionada uma opção (-m) que é usada para incluir uma mensagem explicativa das alterações que foram introduzidas ao repositório. Se escrever apenas `git commit` será enviado para um editor de texto (e.g. Vim) onde terá de escrever a mensagem a associar ao *commit*.

É boa prática adicionar uma mensagem clara em cada *commit*.

Deve realizar um *commit* sempre que faça alterações ao seu código que em conjunto formem uma modificação coerente. Os seguintes exemplos podem originar novos commits:

- adicionar uma nova função;
- adicionar um nova funcionalidade;
- corrigir um bug;

Note que depois do *commit* o código está no repositório, mas apenas na sua cópia local.

4.4 Sincronização de repositórios

O **Git** permite sincronização de cópias de repositórios diretamente em vários dispositivos. Para o efeito do trabalho prático, vamos no entanto assumir uma forma centralizada, utilizando como referência central o repositório disponível online no GitLab.

Para enviar a versão da sua cópia local para o repositório central, pode executar o comando:

```
git push
```

Para receber as modificações mais recentes do repositório central, pode executar o comando:

```
git pull
```

Por exemplo, se um Aluno 1 quiser enviar a sua versão do repositório para um Aluno 2, devem executar a seguinte sequência de passos:

1. Aluno 1 faz *pull* para receber quaisquer atualizações mais recentes. Caso não haja novas atualizações o **Git** reporta que a cópia local está up-to-date.
2. Se estiver up-to-date, Aluno 1 faz *add* e *commit* das suas modificações locais. Pode usar o *status* para verificar o seu estado atual.
3. Quando tiver tudo preparado, Aluno 1 faz *push* das suas modificações.
4. Aluno 2 faz *pull* para receber a versão mais recente enviada pelo Aluno 1.

¹A executar após o `add`

4.5 Modificação de pastas e ficheiros existentes

Assumamos que o Aluno 1 acrescentou e submeteu o ficheiro `CREDITOS.md`, e que o Aluno 2 de seguida faz `git clone` do repositório pela primeira vez, ou faz `git pull` para atualizar a sua versão local. De seguida, o Aluno 2 edita o ficheiro `CREDITOS.md` para acrescentar o seu número e nome.

O seguinte comando confirmará que fez uma modificação localmente:

```
$ git status
On branch master
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git checkout -- <file>..." to discard changes in working directory)
        modified:   CREDITOS.md
no changes added to commit (use "git add" and/or "git commit -a")
```

O Aluno 2 poderá então efetuar e submeter as suas alterações para o repositório central:

```
$ git commit -m "Alterado o ficheiro CREDITOS.md"
$ git push
```

4.6 Log

O comando `git log` lista o histórico de versões. Depois das alterações acima mencionadas, obtemos:

```
$ git log
commit 68be25ae3ab33f1c4f781ac21042a9253372dd58 (HEAD -> master)
Author: omp <omp@di.uminho.pt>
Date:   Sat Nov 2 11:26:26 2024 +0000
```

Alterado o ficheiro `CREDITOS.md`

```
commit 3325b65bbbbb38e9eb5621d10dbae763d1321850f
Author: omp <omp@di.uminho.pt>
Date:   Sat Nov 2 11:21:27 2024 +0000
```

Adicionado o ficheiro `CREDITOS.md`

Há uma grande variedade de opções para este comando. Para mais detalhes pode consultar: <https://git-scm.com/book/en/v2/Git-Basics-Viewing-the-Commit-History>.

Referências

Para informação mais detalhada sugere-se a consulta de documentação do `Git`, nomeadamente:

<https://git-scm.com/doc>